

Training a Model with Gradient Descent

Visualising $J(w, b)$, the Descent Algorithm, and the Learning Rate

Rokan Uddin Faruqui
Professor, Department of CSE

University of Chittagong
rufaruqui@cu.ac.bd

Week 1 · Part 2

Where we left off, and where we are going

Part 1 gave us

$$f_{w,b}(x) = wx + b$$

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2$$

$$\text{Goal:} \quad \min_{w,b} J(w,b)$$

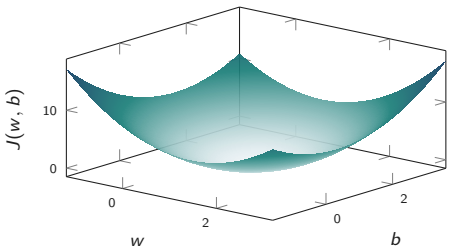
We can *state* the goal. We cannot yet *reach* it: trying every (w, b) is impossible — the parameter space is continuous.

Today

1. See J as a surface and as contours
2. Gradient descent: the algorithm
3. Why the derivative points the right way
4. The learning rate α
5. Derive the gradients for linear regression
6. Batch gradient descent in action

Visualising the Cost Function

With two parameters, J is a surface

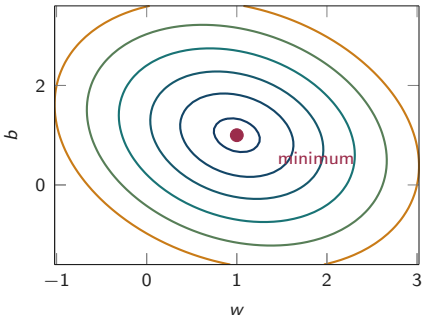


- Horizontal axes: the *parameters* w and b.
- Vertical axis: the cost J — how badly that line fits the *fixed* training data.
- A **convex bowl**: one lowest point, no local minima, no plateaus.
- Each point on the surface is one candidate line $f_{w,b}$.

Key idea

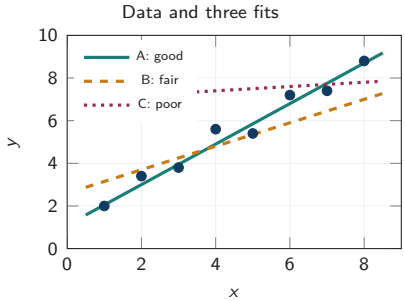
Training searches for the bottom of this bowl, blindfolded: we feel the local slope but never see the whole surface.

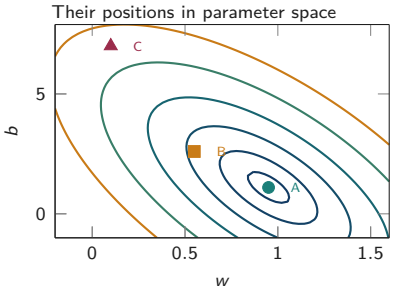
The same surface, seen from above: contour plots



- Each closed curve joins all (w, b) pairs having the *same* cost — exactly like elevation lines on a topographic map of Chittagong Hill Tracts.
- The centre of the innermost ellipse is the minimum.
- Contours *close together* $\Rightarrow$ steep slope; *far apart* $\Rightarrow$ nearly flat.
- Elongated ellipses signal that the two parameters have very different scales — this motivates feature scaling in Week 2.

Reading the contour map: three candidate lines





A sits at the centre (low cost); C sits on an outer contour (high cost). **Better line ⇔ inner contour.**

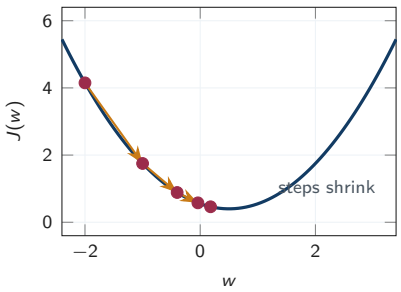
Gradient Descent

The idea: walk downhill

You are standing on a hillside in dense fog. You want the valley floor. A reasonable procedure:

1. Feel the ground around your feet.
2. Identify the direction of *steepest descent*.
3. Take a small step that way.
4. Repeat until the ground is flat.

That procedure is **gradient descent**. The "feel the ground" step is a derivative; the "small step" is scaled by the learning rate.



Gradient descent for two parameters

Start from some initial w, b (commonly $w = 0$, $b = 0$). Repeat until convergence:

$$\begin{cases} \text{tmp}_w := w - \alpha \dfrac{\partial}{\partial w} J(w, b) \\ \text{tmp}_b := b - \alpha \dfrac{\partial}{\partial b} J(w, b) \\ w := \text{tmp}_w, \qquad b := \text{tmp}_b \end{cases}$$

- $\alpha > 0$ is the **learning rate**, typically 0.001–0.3; it controls the step size.
- $:=$ denotes assignment, not a mathematical equality.
- The minus sign is what makes it *descent*: we move *against* the gradient, the direction of steepest increase.

Simultaneous update — a correctness requirement

Correct: simultaneous

$$\begin{aligned} \text{tmp}_w &:= w - \alpha\, \partial_w J(w, b) \\ \text{tmp}_b &:= b - \alpha\, \partial_b J(w, b) \\ w &:= \text{tmp}_w \\ b &:= \text{tmp}_b \end{aligned}$$

Both derivatives are evaluated at the *same* point (w, b) — the old one.

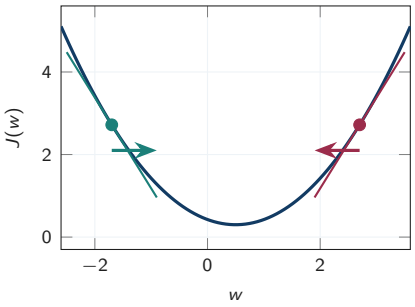
Incorrect: sequential

$$\begin{aligned} w &:= w - \alpha\, \partial_w J(w, b) \\ b &:= b - \alpha\, \partial_b J(\underbrace{\quad w \quad}_{\text{already changed!}}, b) \end{aligned}$$

The second derivative is taken at a *different* point. This is another algorithm; it sometimes still converges, which makes the bug hard to spot.

Common pitfall

In NumPy, `w -= alpha*dj_dw` then `b -= alpha*dj_db` is safe *only if* both gradients were computed *before* either assignment.

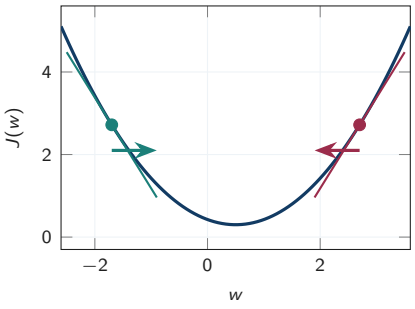


Right of the minimum (rose): the tangent rises, so $\frac{dJ}{dw} > 0$. Then

$$w := w - \alpha \cdot (\text{positive}) \;\Rightarrow\; w \text{ decreases.}$$

We move *left*, toward the minimum.

Why the derivative points the right way



Right of the minimum (rose): the tangent rises, so $\frac{dJ}{dw} > 0$. Then

$$w := w - \alpha \cdot (\text{positive}) \;\Rightarrow\; w \text{ decreases.}$$

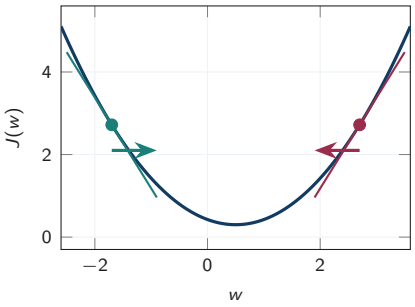
We move *left*, toward the minimum.

Left of the minimum (teal): the tangent falls, so $\frac{dJ}{dw} < 0$. Then

$$w := w - \alpha \cdot (\text{negative}) \;\Rightarrow\; w \text{ increases.}$$

We move *right*, toward the minimum.

Why the derivative points the right way



Right of the minimum (rose): the tangent rises, so $\frac{dJ}{dw} > 0$. Then

$$w := w - \alpha \cdot (\text{positive}) \;\Rightarrow\; w \text{ decreases.}$$

We move *left*, toward the minimum.

Left of the minimum (teal): the tangent falls, so $\frac{dJ}{dw} < 0$. Then

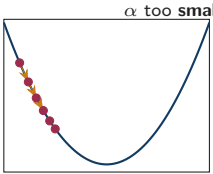
$$w := w - \alpha \cdot (\text{negative}) \;\Rightarrow\; w \text{ increases.}$$

We move *right*, toward the minimum.

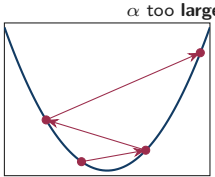
At the minimum: $\frac{dJ}{dw} = 0$, so $w := w - 0$. Gradient descent leaves a minimum alone — it is a fixed point.

The Learning Rate

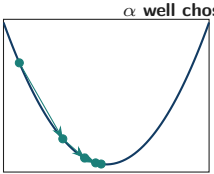
Choosing α: too small, too large, just right



Converges, but needs a huge number of iterations. Training is slow, not wrong.



Overshoots the minimum each step; *J increases*. Diverges.

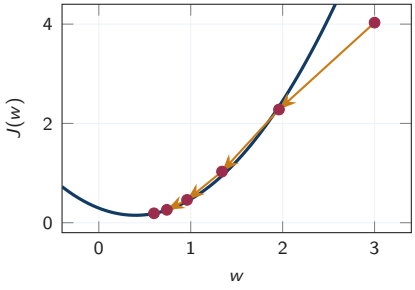


Rapid, monotone decrease in J.

Key idea

Try $\alpha \in \{0.001, 0.003, 0.01, 0.03, 0.1, 0.3\}$ — roughly $3\times$ steps. Plot J against iteration number for each and pick the largest α that still decreases J smoothly.

Steps shrink automatically — α need not



As we approach the minimum the slope $\left|\partial J/\partial w\right|$ gets smaller, so the step

$$\Delta w = -\alpha \frac{\partial J}{\partial w}$$

gets smaller too — *with α held fixed*.

Key idea

Gradient descent decelerates near a minimum on its own. For a convex problem like linear regression, α need not be decayed by hand.

For non-convex problems a fixed α can stall in a *local* minimum — Module 2 introduces Adam.

Gradient Descent for Linear Regression

Deriving the two partial derivatives

With $f_{w,b}(x) = wx + b$ and $J(w,b) = \dfrac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2$:

$$\frac{\partial J}{\partial w} = \frac{1}{2m}\sum_{i=1}^{m} 2\left(wx^{(i)} + b - y^{(i)}\right)\cdot\frac{\partial}{\partial w}\left(wx^{(i)} + b - y^{(i)}\right) = \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)x^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{1}{2m}\sum_{i=1}^{m} 2\left(wx^{(i)} + b - y^{(i)}\right)\cdot 1 = \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)$$

Key idea

This is where the $\frac{1}{2}$ in J pays for itself: the 2 from the chain rule cancels it exactly. The $\frac{1}{2}$ changes the *value* of J but not its *minimiser*.

Note the structure: both gradients are averages of the residual $\left(\hat{y}^{(i)} - y^{(i)}\right)$ — weighted by $x^{(i)}$ for w, unweighted for b.

Batch gradient descent for univariate linear regression

Initialise $w := 0$, $b := 0$. Repeat until convergence:

$$w := w - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)} \right) x^{(i)}$$

$$b := b - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)$$

(updated simultaneously)

"Batch" means each update uses *all* m examples; mini-batch and stochastic variants use a subset per step.

Why we are safe here

The squared-error cost of a linear model is *convex* in (w, b) — a single bowl. With a suitable α, gradient descent always reaches the *global* minimum, whatever the initialisation.

One iteration by hand

Data $(1,1),(2,2),(3,3)$, $m=3$. Start at $w=0$, $b=0$, take $\alpha=0.1$.

Step 1 — predictions and residuals. $\hat{y}=(0,0,0)$, so residuals $\hat{y}^{(i)}-y^{(i)}=(-1,-2,-3)$.

Step 2 — gradients.

$$\frac{\partial J}{\partial w}=\frac{1}{3}\big[(-1)(1)+(-2)(2)+(-3)(3)\big]=\frac{-14}{3}\approx -4.667, \qquad \frac{\partial J}{\partial b}=\frac{-1-2-3}{3}=-2.$$

Step 3 — simultaneous update.

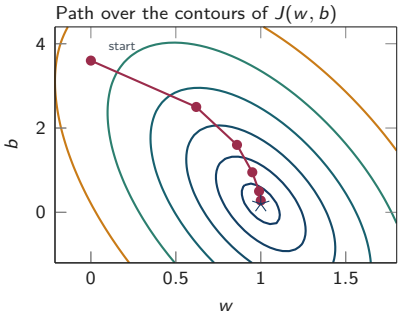
$$w:=0-0.1(-4.667)=0.467, \qquad b:=0-0.1(-2)=0.2.$$

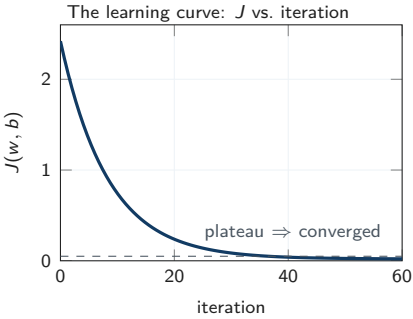
Check. Cost fell from $J(0,0)=\frac{1}{6}(1+4+9)=2.33$ to $J(0.467,0.2)\approx 0.47$. Both parameters moved toward the true $(w,b)=(1,0)$; b will be pulled back down in later iterations.

Common pitfall

Note that b overshot *upward* on iteration 1 even though its optimum is 0. Individual parameters may move away from their optimum; only J is guaranteed to decrease.

Running gradient descent: the trajectory





Always plot the learning curve

A correct gradient descent decreases J on *every* iteration. If J ever rises, α is too large or the gradient code has a bug.

Declaring convergence

Automatic test

Stop when J decreases by less than ε (e.g. 10^{-3}) in one iteration.

Drawback: the right ε varies wildly across problems and is hard to guess in advance.

Visual test — preferred

Plot J against iteration count and look for the curve flattening. You also see divergence, oscillation and a badly chosen α immediately.

Debugging checklist

- J increasing $\Rightarrow$ reduce α by 3–10$\times$.
- J oscillating $\Rightarrow$ α slightly too large.
- J barely moving $\Rightarrow$ α too small, or features on wildly different scales (Week 2).
- Sanity check: with a very small α, J *must* decrease every iteration. If it does not, the bug is in the gradient, not in α.

Wrap-up and Assessment

Summary of Week 1

The three-part recipe

1. **Model:** $f_{w,b}(x) = wx + b$
2. **Cost:** $J(w, b) = \frac{1}{2m} \sum_i (f_{w,b}(x^{(i)}) - y^{(i)})^2$
3. **Optimiser:** gradient descent

$$\theta := \theta - \alpha \, \partial_\theta J$$

Every algorithm in this course — logistic regression, neural networks, recommenders — is a different choice of these same three parts.

Things to remember

- Update w and b *simultaneously*.
- The derivative's *sign* steers; α sets the *size*.
- Squared-error + linear model $\Rightarrow$ convex $\Rightarrow$ global minimum guaranteed.
- "Batch" = all m examples per step.
- Plot J vs. iteration, always.

Next week

Multiple features, vectorisation, feature scaling, and choosing α systematically.

Descriptive questions

1. Explain what a contour plot of $J(w, b)$ represents. What does the spacing between adjacent contours tell you about the gradient there? **[5]**
2. Write the gradient descent update rule for (w, b) and explain why the updates must be simultaneous. Show, with a small example, how a sequential update yields a different trajectory. **[6]**
3. Using the sign of dJ/dw, argue that the update $w := w - \alpha\, dJ/dw$ moves w toward the minimum whether the current w lies to the left or to the right of it. **[5]**
4. Derive $\partial J/\partial w$ and $\partial J/\partial b$ for univariate linear regression with the squared-error cost. State clearly where the chain rule is applied. **[8]**

5. Describe the behaviour of gradient descent when α is (a) too small, (b) too large. Sketch the corresponding curve of J against iteration number for each case. **[5]**
6. Explain why gradient descent takes smaller steps near the minimum even though α is held constant. **[4]**
7. For the data $(1,1),(2,2),(3,3)$, model $f_{w,b}(x) = wx + b$, initial $w = b = 0$ and $\alpha = 0.1$, carry out *two* iterations of gradient descent by hand, reporting w, b and J after each. **[8]**
8. Why is the global optimum guaranteed for linear regression with squared error, but not for a neural network? **[4]**

Design questions

1. **Design a learning-rate schedule.** You must train the same model on 200 different districts' data overnight, unattended — no human can watch the learning curves. Design an automated procedure that selects α per district and detects divergence. Specify the decision rule, the quantities you would log, and the failure case your design does not handle. **[12]**
2. **Design a stopping criterion.** Propose a convergence test that is scale-invariant — one that behaves the same whether prices are recorded in taka or in lakh taka. Justify why your criterion has this property, and contrast it with the naive test "stop when $\Delta J < 10^{-3}$". **[10]**
3. **Design a test suite.** A classmate's `compute_gradient` function returns plausible numbers but the model never fits well. Design three unit tests that would localise the bug — for example, tests exploiting a data set whose optimal parameters you know analytically. State what each test's expected output is and which bug it catches. **[12]**

Design questions (continued)

4. **Redesign the optimiser.** Suppose $\partial J/\partial w$ is expensive because $m = 10^8$. Propose a modification to batch gradient descent that reduces per-iteration cost, and analyse the trade-off it introduces in the convergence behaviour. **[10]**

5. **Diagnose from a plot.** You are shown a learning curve in which J falls sharply for 50 iterations, then rises steadily. List the two most likely causes and design an experiment that distinguishes between them. **[8]**

Reading

Primary text

G. James, D. Witten, T. Hastie and R. Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.

→ **§3.1.1–3.1.3** — estimating the coefficients by least squares, and assessing the fit.

Supplementary

C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

→ **§3.1.3** — sequential (gradient-based) learning and the least-mean-squares update rule.

Lab (CSE 816, Week 1): *Gradient Descent* — implement `compute_cost` and `compute_gradient`, then plot the learning curve.

Practice quiz: *Train the model with gradient descent.*